봉수대

Bongsudae

A BMP collector written in Go

The Bongsudae was a beacon tower of old Korea, relaying news from hill to hill.

This one relays news from your routers.



2026-11 · v0.3.0 · in production · open source (ISC)

01 At a glance

BGP routes from your routers, to Kafka and a web API

Routers send their BGP routes over BMP (RFC 7854). Bongsudae turns them into OpenBMP-format Kafka messages and keeps the route tables in memory

1 Compatible

OpenBMP v1.7 message format.

Existing Kafka consumers need no changes

2 Never stops

Only the faulty connection or message is dropped.

Upgrades keep router sessions up

3 Query right away

Route tables per peer · view · family.

Find prefixes with the web API and CLI

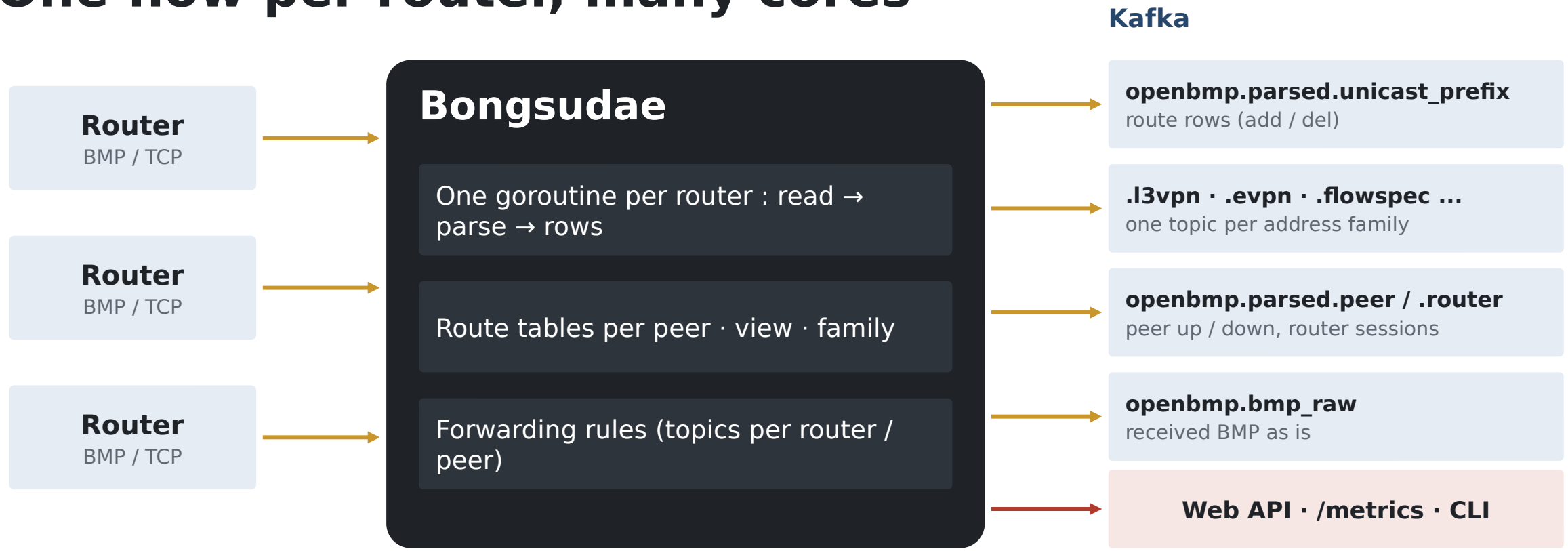
33	1,733	66.8 M	860 M	0
Routers	BGP peers	live routes	BMP messages	errors · Kafka failures

02 Starting point

What we hit in production became the design goals

What we hit (openbmpd 2.2.3)		Bongsudae's goals
One scanner connection ended the whole collector (29 times in a month), each time a full table resync	→	Per connection : a bad connection is closed alone, router slots are kept apart
The same error logged without end : 93 GB of container logs	→	Repeated errors are summarized in one line per kind per minute
One RFC-violating Peer Up from one router: none of that router's routes arrived	→	That message is handled on its own and the session stays up (one log line)
Upgrading meant a restart = full table resync (18 min)	→	Hitless upgrade : sessions and state are handed to the new build

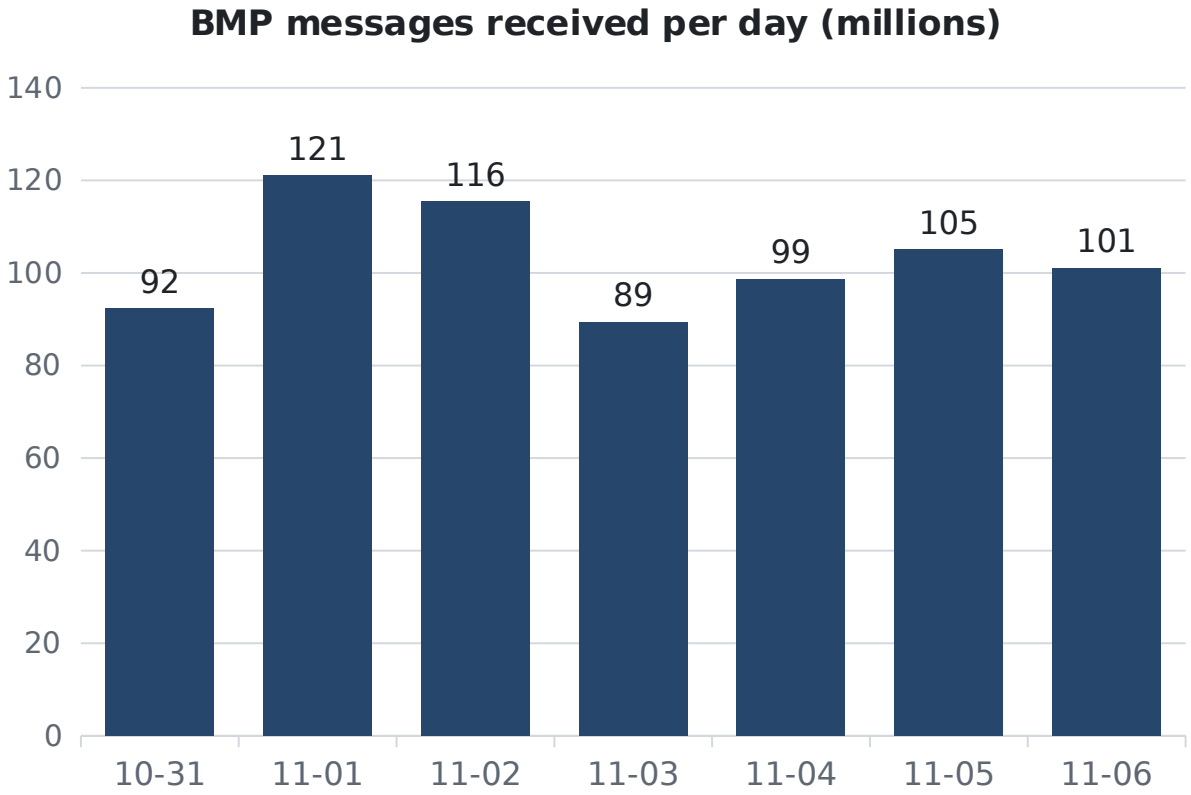
One flow per router, many cores



- **In order within a router, in parallel across routers :** restart simulation (25 routers, 3 M messages) 104 s → 16 s

04 In production

8 days in production: 860 M messages, 0 errors



4 hitless upgrades (no router reconnected)

Build	Routes handed over	Read pause
11-02 0.2.0-dev	77.07 M	9.5 s
11-03 0.2.0-dev	77.21 M	7.3 s
11-03 0.3.0-dev	77.23 M	5.7 s
11-04 0.3.0-dev	77.31 M	5.7 s

- The only resync (18 min) was the switch-over on 10-30
- Zero write failures, even across a Kafka restart
- Routers 25 → 33 (4 vendors), 6 address families

Sum of one-minute statistics (2026-10-30 18:22 - 11-07 14:50). Routes are counted per peer · view · family

05 **Compatible**

Same format, RFC-compliant handling

```
V: 1.7   C_HASH_ID: ...   T: unicast_prefix   L: 385   R: 1
add  1  …  198.51.100.7  …  203.0.113.9  64511  2026-11-06 05:05:36.749  192.0.2.0  24  1  igp   64511 64999 …
action · seq · router · peer · AS · time · prefix · length · IPv4 · origin · AS path · count · origin AS … (32 tab-separated fields)
```

10,901,264

/ 10,901,415 rows identical

80-minute sample. The rest are window edges

9 hours

live side-by-side comparison

7 - 9 M rows per hour, 0 unexplained differences

6 items

handled per the RFCs

with those aligned, 0 different rows (1 hour, 4.76 M UPDATEs)

- **Example per the RFCs :** AS_SET {64500,64501} counts as AS path length 1 (RFC 4271). Unknown extended communities still show their value

06 Never stops

A problem stays where it happens

Situation	What Bongsudae does
Internet scanner connects to the BMP port	Only that connection is closed if no Initiation arrives in 10 s; 4 per address, separate router slots
Peer Up without OPEN (RFC violation)	Session kept, peer up with empty capabilities, one log line
The same error repeats	One line per kind per minute (logs never fill the disk)
Broken message, code bug (panic)	Only that message is dropped (fuzz tested)
Kafka restart, topic re-created	Reconnects and re-reads topic metadata (no restart)
Router reconnects, old session lingers	The new session takes over and the old one is closed
Collector alive but doing nothing	systemd watchdog restarts it within 3 minutes
Huge requests flood the web API	Row cap per request, concurrency slots, deadlines: collection goes on
Config file mistake (typo, bad prefix)	Refuses to start; on reload nothing changes
Upgrading to a new release	Sessions handed to the new binary (no full resync). On failure the old one goes on

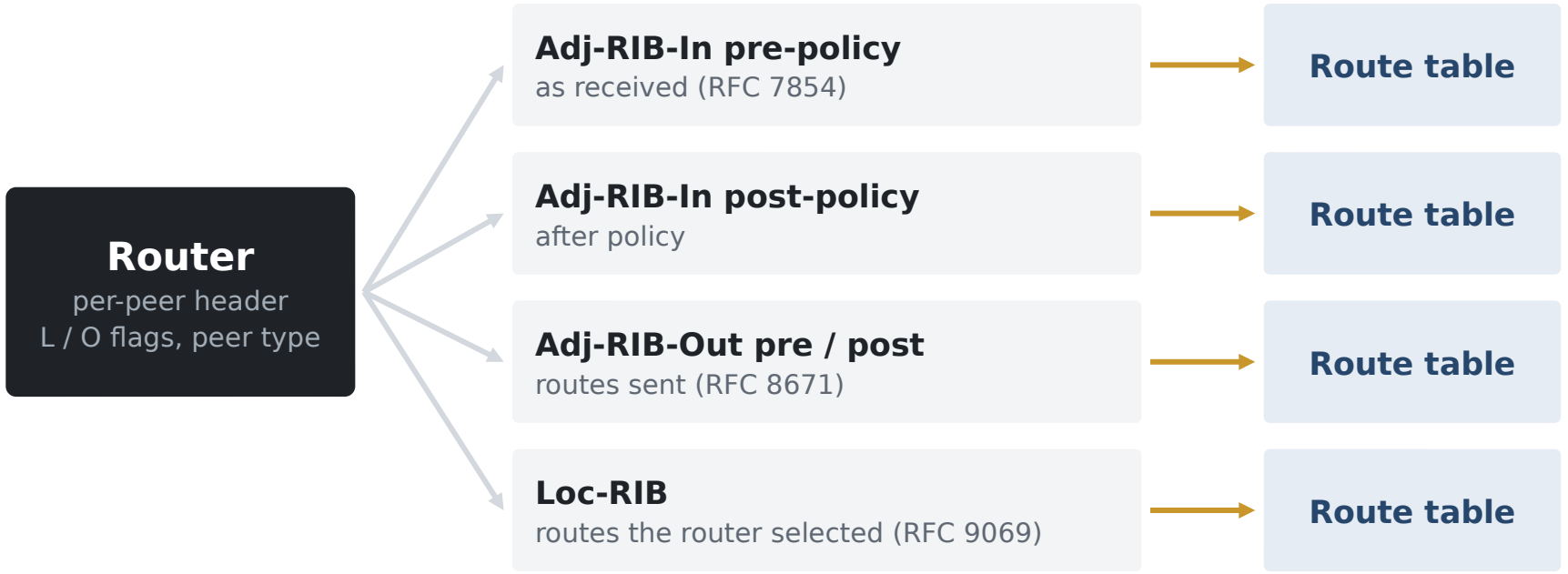
07 Address families

Beyond unicast: one topic per family

Address families	SAFI	Kafka topic	Row format	Route table
IPv4 / IPv6 unicast	1	unicast_prefix	OpenBMP, 32 fields	Yes
Labeled unicast (incl. 6PE)	4	labeled_unicast_prefix	32 fields + labels	Optional
VPNv4 / VPNv6 (L3VPN)	128	l3vpn	OpenBMP, 34 fields	Optional
IPv4 / IPv6 multicast	2	multicast_prefix	32 fields	Optional
EVPN	70	evpn	OpenBMP, 40 fields	No
FlowSpec (IPv4 / IPv6 / VPN)	133 / 134	flowspec	Bongsudae, 43 fields	Optional (rule table)
BGP-LS, SR Policy, MVPN ...	-	(not published)	counted per family	-

- **Topics are created on the first route:** families your routers do not send leave no trace in Kafka
- **Route tables per family (bmp.rib):** keep only what you need in memory. FlowSpec, missing from OpenBMP, follows the same layout

One neighbor, separate views



Add-Path

Several paths for one prefix, kept per path ID (RFC 7911)

Only when both OPENs in the Peer Up advertise it

API : path_id, paths_v4 / v6

In production: one router moved from pre-policy (90 neighbors) to one Loc-RIB → 1.66 M routes, unchanged
Loc-RIB rows in Kafka have peer_ip 0.0.0.0 · API: view=loc-rib

Ask right away, no Kafka consumer needed

- **Status :** router / peer state, route counts, warnings, capabilities, history
- **Prefix lookup :** exact / longest / covering / more, 5,000 per request
- **Hijack check :** routes inside our prefixes whose origin is not our AS (not_origin)
- **Views · families:** view=loc-rib, family=vpnv4, FlowSpec rules
- **Health :** 60 min of 1-minute stats, memory, Kafka. Prometheus at /metrics
- **CLI = API :** arguments map 1:1 to API paths, other servers with -api

```
$ bongsudae -status prefix p=192.0.2.0/22 \
      match=more not_origin=64500
192.0.2.0/22  match=more  not_origin=[64500]  1 / 1
rows

ROUTER       PEER         PREFIX       AS PATH
198.51.100.7 203.0.113.9  192.0.2.0/24  64511 64999

$ curl :5001/api/routers/198.51.100.7
{"state": "up", "peers_up": 12,
 "peers_synced": 3, "peers_settled": 12,
 "routes_v4": 1204513, ...}
```

About 65 bytes per route · identical attributes are shared · large lists copy one peer at a time and never block collection

Addresses and ASes in the example are documentation values (RFC 5737, RFC 5398)

10 Hitless upgrade

Upgrade without the routers noticing

1 Pause

Stop reading routers at a message boundary

2 Flush

Send everything pending to Kafka

3 Hand over

Socket fds + state (peers, sequence, tables) to a file

4 exec

New binary, same pid (systemd sees nothing)

5 Resume

Take the sockets, read on. Tables load in the background

Before (restart) : every router reconnects and resends its full table for 18 min; monitoring lags meanwhile

Now (-upgrade) : router reads pause for a few seconds. TCP sessions and Kafka sequence numbers carry on

5.7~9.5 s

read pause, 4 production upgrades (77 M routes)

31 upgrades

test : repeated upgrades under load. 0 lost / duplicate rows

bongsudae -upgrade (no sudo). On failure the old build keeps running; 0.2 ↔ 0.3 both ways

Tested on the full production table first

Output: same full table into old and new

```
$ tools/regress.sh
unicast_prefix : 78,170,006 rows
  same 78,170,006, different 0
router 25 · peer 1,331 : different 0
verdict : same (incl. seq)
```

Upgrade: 31 times under load

```
$ tools/handover_test.sh   # 25 fake routers
unicast_prefix : 8,146,379 rows
  lost 0, duplicated 0, reordered 0
routes 8,002,622 · peer 1,331 : same
result : SAME
```

`go test -race`

BMP sample bytes, rules, JSON, fuzzing

→

`regress`

two builds, row by row, on a production RIB dump

→

`handover_test`

old ↔ new build, alternating 31 times

→

`production`

-upgrade, check the 1-minute stats

v0.3.0 : new families and views, yet unicast output is byte-identical to production (2026-11-06)

One binary, one config file

1

binary: download and copy (linux amd64 / arm64)

1

external library (Kafka client franz-go)

16,300

lines of Go code (with tests)

0

C libraries (static build: any distribution)

```
# from GitHub Releases
$ tar xzf bongsudae_0.3.0_linux_amd64.tar.gz
$ sha256sum -c --ignore-missing SHA256SUMS

# /etc/bongsudae/bongsudae.conf
[bmp]
allow = 192.0.2.0/24     # router range
rib   = ipv4-unicast, ipv6-unicast, vpnv4
```

Session reset	Close a router's BMP session via the API; it resends all
Kafka rules	Pick topics per router / peer, or send nothing
bmp_raw output	Raw BMP: feed other collectors, shadow-test builds
Config reload	systemctl reload: allow list etc. Errors change nothing

Compared with other BMP collectors

	Bongsudae	openbmpd	goBMP	pmacct (pmbmpd)
Language	**Go**	C++	Go	C
Kafka messages	**OpenBMP v1.7 (TSV) or JSON**	OpenBMP v1.7 (TSV)	JSON	JSON / Avro
openbmpd consumers work as is	**Yes (checked row by row)**	(original)	No	No
Address families	**unicast, VPN, EVPN, FlowSpec ...**	unicast, L3VPN, EVPN, BGP-LS	unicast, VPN, EVPN, BGP-LS, SR Policy, FlowSpec	Many
Route table in the collector	**Yes (HTTP API queries)**	No (DB consumer builds it)	No (forwards only)	Yes (periodic dumps)
Hitless upgrade	**Yes**	no (restart = resync)	not documented	not documented
License	**ISC**	EPL-1.0 / BSD	Apache-2.0	GPL
Latest release	**0.3.0 (2026-11)**	None since 2022-10 (2.2.3)	Active	Active

Based on each project's public documents and repository, 2026-10. Bongsudae has no BGP-LS / SR Policy yet

v0.3.0 is out; next, easier to use



v0.1 · 2026-11-01

Go public

- openbmpd retired (10-30)
- systemd, one config file
- Published on GitHub (ISC)

v0.2.0 · 2026-11-03

Never stop

- Hitless upgrade
- Kafka key buckets, JSON
- Prometheus /metrics, watchdog

v0.3.0 · 2026-11-06

More families

- L3VPN, EVPN, FlowSpec …
- Views incl. Loc-RIB, Add-Path
- Binary downloads, API limits

Next

Easier to use

- CI on every push
- Grafana dashboard example
- Tests with BMP from GoBGP / FRR

0.2.0 → 0.3.0 : same topics and message format, hitless with bongsudae -upgrade

봉수대

Bongsudae

Compatible · Never stops · Query right away

A BMP collector in Go · OpenBMP v1.7 compatible messages · ISC-licensed open source

https://github.com/yunsog/bongsudae